SOFTWARE FRAMEWORKS – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

Contents

- Preliminaries
- Programming vs Software Engineering
- Generic functionalities in Software Development
- Frameworks: what are they?
- How does Frameworks help with the Software Engineering Approach?
- Framework vs. Libraries
- Why use frameworks?
- Advantages of using frameworks
- Limitations of using frameworks

Preliminaries

- What is this course?
  - **PAF (IT3030)** introduces the fundamentals of full stack web development with an emphasis on application of good software engineering practices and Industry standards.
- Learning Outcomes of this course?
  - After completing this module, you will,
    - Know the basics of Software Frameworks
    - Apply Industry standard software development best practices
    - Understand the REST architectural style
    - Develop Java based REST web services (Back-end)
    - Develop JavaScript based Interactive web applications (Front-end)
- Module Outline
- Assessments

Programming vs Software Engineering

- What is Programming?
  - Programming simply means writing Code to create an application.
- What is Software Engineering?
  - Software engineering is the end-to-end process of applying good engineering principles and software development practices to produce quality software.
- Programming vs Software Engineering
  - Software Engineering is **NOT** just coding. Coding is just one part of a large process.
  - **Q: What other practices come under software engineering?**

Programming vs Software Engineering

- What other practices come under software engineering?
  - Analyzing user requirements and designing the software to meet those requirements.
  - Communicating with all the stakeholders to resolve issues and update them on the progress.
  - Choosing the software architectures, patterns, algorithms, coding techniques, technologies used to realize the software design based on sound design considerations.
  - Quality Assurance (QA) of the developed software to ensure that the software meets the client requirements and acceptable reliability, performance and security considerations.
  - Deploying/ releasing the software for actual use.
  - Maintaining the developed software as needed after release/ deployment.

Generic functionalities in Software Development

- Generic functionalities in Software Development
  - Most (web) applications have a lot of features in common.
    - Eg: User Authentication, routing, Database connectivity, Scheduled jobs etc.
  - There is little point in rewriting all the functionalities (including the common ones) from scratch every time new software is developed.
  - What if the generic functionalities can be collected and made reusable?

Frameworks: what are they?

- A framework is an integrated set of software artifacts (such as classes, objects, and components) that collaborate to provide a reusable architecture for a family of related applications [1].
- A framework enables a coding environment that contains low-level libraries to address conventional coding issues. The objective of a framework is to deliver faster development of an application. This includes everything we need to build large-scale applications, such as templates based on best practices [4].
- A framework is a foundation for developing software applications. Software engineers and developers use a framework as a template to create websites and applications. Developers do this by adding code to a framework, then personalizing it for their specific purpose [5].
- A software framework is an abstraction in which software, providing generic functionality, can be selectively changed (**extended**) by additional user-written code, thus providing application-specific software [6].

How does Frameworks help with the Software Engineering Approach?

- How does Frameworks help with the Software Engineering Approach?
  - Architectural Patterns (eg: MVC, MVVM)
  - Design Patterns
    - Inversion of Control
  - Standards/ Best Practices (Naming conventions, Coding standards, Directory structures etc.)
  - Tooling Support (Package managers, Dependency managers, Boilerplate code generation)
  - Testing Support

Framework vs. Libraries

- Certain features make a framework different from other library forms, including the following [7]:
  - Default Behavior: Before customization, a framework behaves in a manner specific to the user's action.
  - Inversion of Control: Unlike other libraries, the global flow of control within a framework is employed by the framework rather than the caller.
  - Extensibility: A user can extend the framework by selectively replacing default code with user code.
  - Non-modifiable Framework Code: A user can extend the framework, but not modify the code.

Why use frameworks?

- The designers of software frameworks aim to facilitate software developments by allowing designers and programmers to **devote their time to meeting software requirements rather than dealing with the more standard low-level details of providing a working system, thereby reducing overall development time** [6].
- The aim of frameworks is to provide a common structure so that developers don't have to redo it from scratch and can reuse the code provided. In this way, frameworks allow us to cut out much of the work and save a lot of time.
- To summarize: there's **no need to reinvent the wheel**.

Advantages of using frameworks

- Improved coding, easy code reusability and ease of debugging code compared with from-scratch developments.
- Community support.
- Accelerated development (if the programmer is familiar with the framework).
- Often provides caching and optimized processes for resource intensive tasks.
- Enables faster methods for development with less code (boilerplate code etc.).
- Better compliance to accepted security standards.

Limitations of using frameworks

- Learning the Framework and not the programming language prevent programmers from gaining an in-depth understanding of the programming language.
- Users of a framework are forced to respect the limitations and conventions imposed by its design.
- Options to tweak functionalities are limited.
- A framework will come with everything required to satisfy a wide range of use cases, not all these features will be used for a given project.
- The right framework for the application should be chosen, or else performance and user experience may be impacted negatively.
- We must be up-to-date with new/deprecated features in every version.

Some examples

- Spring Boot (Java)
- Express.js (with Node.js)
- Laravel (PHP)
- .NET Core (C# etc.)
- Angular (Typescript)
- Vue.js (JavaScript)

References

1. Software Engineer vs. Programmer: What's the Difference?
2. Frameworks: Why They Are Important and How to Apply Them Effectively
3. What is a Web Framework, and Why Should I use one?
4. The Difference Between a Framework and a Library
5. What Is a Framework? (Definition and Types of Frameworks)
6. Software framework
7. Techopedia Explains Software Framework

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

VERSION CONTROLLING & WORKFLOWS WITH GIT - I

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

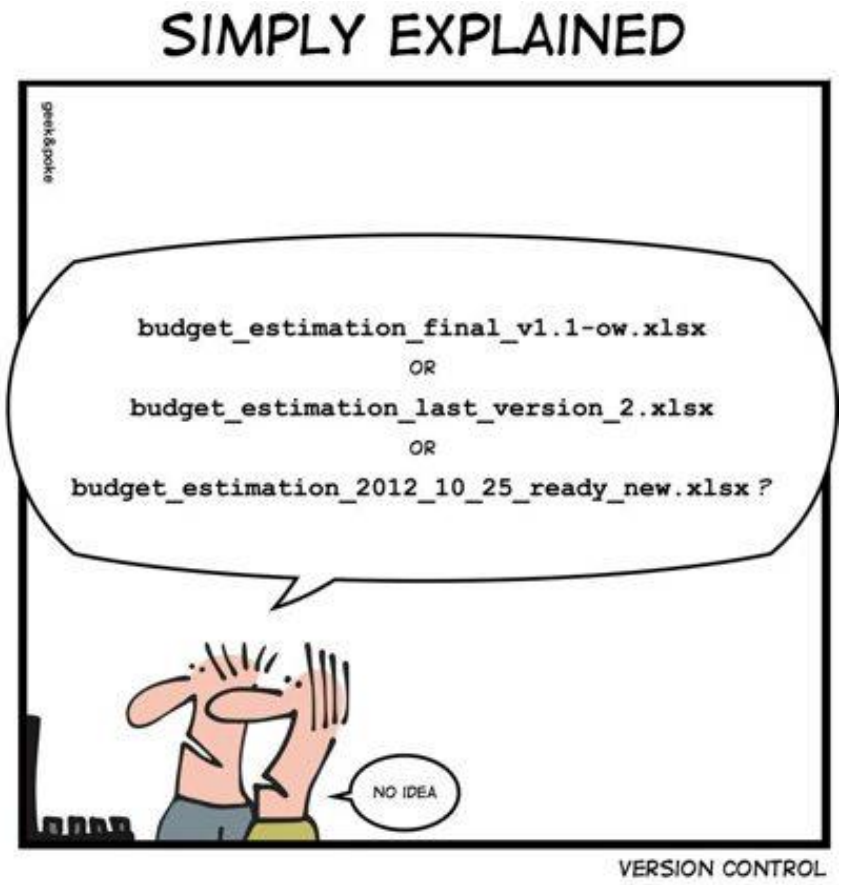
Contents

- Version Controlling: What?
- What you know about Version Controlling
- Version Controlling: Scenario Discussion
- A bit of history on Version Controlling Systems (VCS)
- Git: the de-facto standard today in VCS
- Git Basic Concepts
- Conclusion

Version Controlling: What?

- “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.” [1].

What you know about Version Controlling

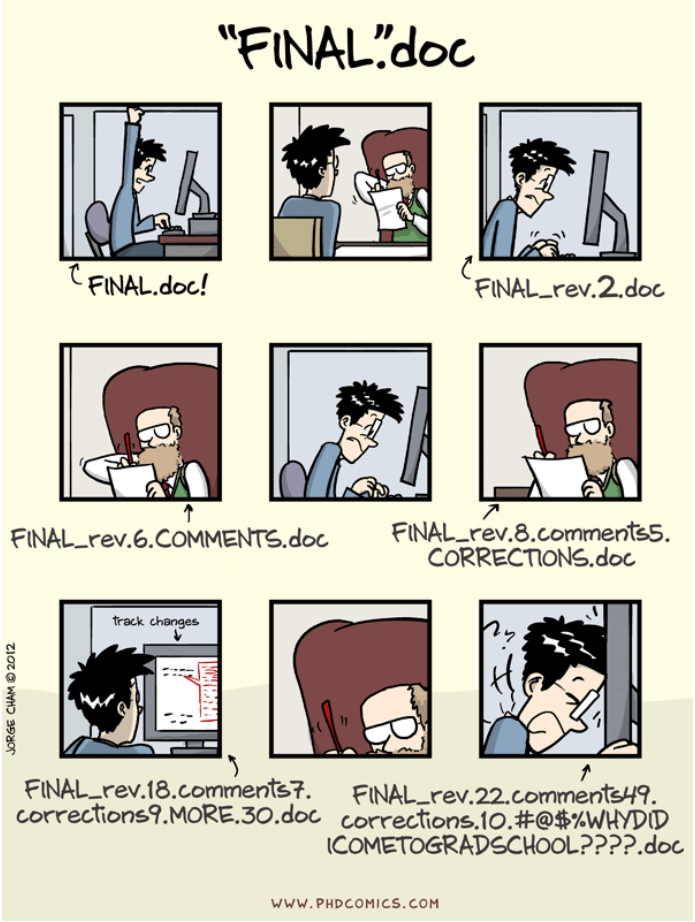


Source: https://www.datamation.com/trends/tech-comics-version-control/

What you know about Version Controlling

- Q: What methods of Version Controlling are you familiar with?
  - Keeping multiple copies of files with different names
  - Google Drive/ OneDrive
  - Undo/ Redo Buffer

- Now, let's see how one student named Amith does version controlling.



Source: https://swcarpentry.github.io/hg-novice/01-basics/

Version Controlling: Scenario Discussion

- Suppose Amith is working on one file called numbers.py for an assignment.
- Amith has worked on the file for several days now, and has saved multiple copies of the file, i.e., numbers1.py, numbers2.py, numbers3.py and numbers.py.
- numbers.py is the latest version of the file. Whenever a new update is to be made to the file, first the existing numbers.py is renamed to numbersX.py (X = {1, 2, 3, 4…}) and then the numbers.py is saved with the most recent change.
- Amith needs to submit the assignment now, but at the last moment he discovers that the numbers.py (most recent version) throws a serious error.

Version Controlling: Scenario Discussion

- **Q1:** What can Amith do?
- **A:** Amith can submit the one before (e.g.: numbers10.py).

- **Q2:** What if the most recent version (numbers.py) has some good changes too, which Amith thinks needs to be in the submitted file?
- **A:**
  - Compare numbers10.py and numbers.py and Copy the good parts.
  - Paste the copied code as fit in the appropriate places in numbers10.py.
  - Replace the numbers.py with this modified file.

Version Controlling: Scenario Discussion

- The assignment was submitted. Now, Amith needs to keep this numbers.py and all its previous versions for future submissions too.
- However, Amith has a nagging feeling about what will happen if the HDD of his laptop fails.

- **Q3:** What can Amith do address this very valid concern?
- **A:** Copy all the versions of numbers.py to a Cloud storage location (OneDrive/ Google Drive).

Version Controlling: Scenario Discussion

- What are the pros of above approach?
  - All the versions are safely stored.
  - The files can be accessed from anywhere now.
- The cons?
  - Considering accessing from anywhere: If Amith develops two versions of numbers.py (both named the same), one at the computer lab in his university (then uploaded to the cloud) and another numbers.py separately on his laptop, Amith runs the risk of replacing the version on laptop with the other if he uploads the first file without the name being changed when his laptop syncs with the cloud storage.

Version Controlling: Scenario Discussion

- All in all, Amith now has a working Version Controlling System.
- His system can,
  - **Revert** to a past version.
  - **Compare** two file versions.
  - **Push** changes done from anywhere to a secure remote location on the cloud.
  - **Pull** changes from the cloud to anywhere.
  - **Merge** different versions of files.

Version Controlling: Scenario Discussion

- Now, Binesh joins Amith as the future assignments are to be done as pair work.
- It is more important than ever to follow a strict discipline when using Amith's system of version controlling as there are other people involved (than Amith).
- However, just focusing on naming the files correctly (to avoid overwrites) is not enough.
- Just seeing the files will not give each developer a high-level idea about what each other is doing.

Version Controlling: Scenario Discussion

- Solution?
  - A log file!
- How?
  - The log file should contain information on **who** did **what when** so everybody would be aware of what is happening.

Log - Notepad

File Edit Format View Help

```
| EntryNo | DateTime | Author | Change                                                          |
+---------+----------+--------+-----------------------------------------------------------------+
| 01      | 08/02/23 | Amith  | Added numbers.py file and all previous versions to OneDrive.|
| 02      | 09/02/23 | Amith  | Added comments on Add() method on numbers.py.                   |
| 03      | 10/02/23 | Binesh | Added new file sequence.py to OneDrive folder.                  |
```

Version Controlling: Scenario Discussion

- Does Amith's method solve the problem of version controlling he had earlier?
  - Yes.
- Does that mean it is an easy system to follow?
  - Not really. There are several specific rules the users of the system must follow in order to make it work.
  - As the number of users of this system increase, so does the chance of someone making a mistake and ruining the system for everybody.
- Should he follow this approach then?
  - Yes, it's a good system. But the management of the system should not be given to the users. It should be automated.
- Are there such automated systems already?
  - Yes! That's what version controlling systems like GIT, SVN and Mercurial do.

A bit of history on Version Controlling Systems (VCS)

- The first proper Version Control System (VCS), named *Source Code Control System (SCCS)* was released in 1973 by Bell Labs.
- It was a first-generation VCS, which are now collectively known as **Local Version Control Systems**.
- Local VCS were intended to track changes for individual files and checked-out files could only be edited locally by one user at a time.
- Facilitating collaboration among large teams was problematic.

A bit of history on Version Controlling Systems (VCS)

- To address the problem of collaboration, a new type of VCS were developed.
- This second generation of VCS are collectively known as **Centralized Version Control Systems.**
- Centralized VCS allowed users to refer files stored on a centralized repository through a network and allowed multiple users to use the files concurrently.
- Follows a *Client-Server* approach.
- However, they would all have to commit their code to the centralized repository, which would require network access.
- A central authority is required to maintain the Centralized repository.

A bit of history on Version Controlling Systems (VCS)

- With the advent of open-source software came the third generation of VCS. They are collectively known as **Distributed Version Control Systems**.
- The earlier Centralized VCS required a central authority to maintain the centralized repository. No such authorities are usually found in open-source projects.
- Instead, there are just contributors who may develop features which may or may not be relevant to the main open-source project.
- Therefore, Distributed VCS have *peer-to-peer (P2P)* approach to version controlling.
- Distributed VCS are widely used today.
- De-facto standard in Distributed VCS now?
  - **Git!**

Git: the de-facto standard today in VCS

- Created by Linus Torvalds (creator of Linux) in 2005.
- Pronounced as *Ghi-T*. Not *Ji-T (*A "git" is a cranky old man. According to online sources, Linus meant himself).
- Originally designed to do version control on Linux kernel.
- Git supports non-linear development (thousands of parallel branches) and is fully distributed.

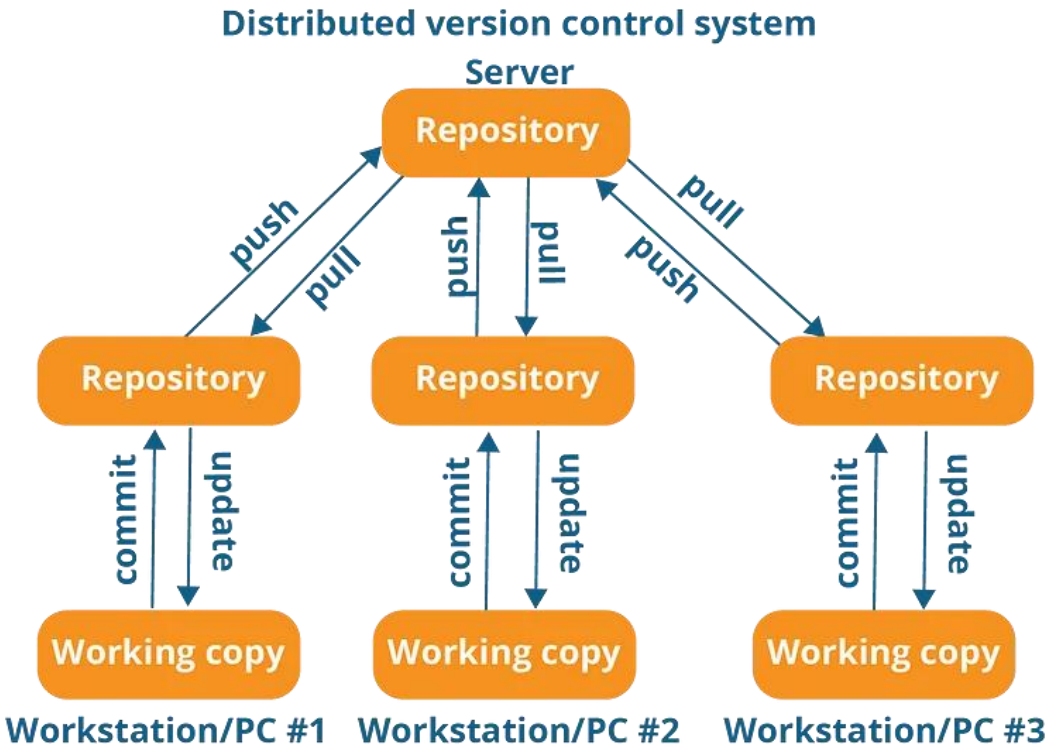
Git Basic Concepts

- As mentioned, git does not necessarily need a Centralized Repository to work (It can work in somewhat centralized manner too, which we will see).
- Every user maintains a **local repository** of their own.
- When starting out they will obtain this local repository,
  - from another user or,
  - they may **clone** it from a central location or,
  - They may even initialize a brand-new repository from scratch.
- This repository will contain what others have done originally, what the user has done themselves and even work of others as selected by the user.
- They can **commit** and **update** their local repository without any interference from any others.

Git Concepts

- Git has two types of repositories.
  - Local repositories – which is owned by each user.
  - Remote repository – a centralized location accessible to all users. A single source of truth.
    - The users will **push** their new updates to this repository so those can be shared with others as well.
    - The users will **pull** new updates from others from this repository.
  - Why do we need Remote repositories?
    - Scenario Discussion: The problem with Distributed Version Controlling. Which project is the "right" project when multiple versions of the project exist?
    - Solution? Remote repositories as a single source of truth!
  - Git != GitHub (Git is not Github)!

Git Concepts



Source: https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383

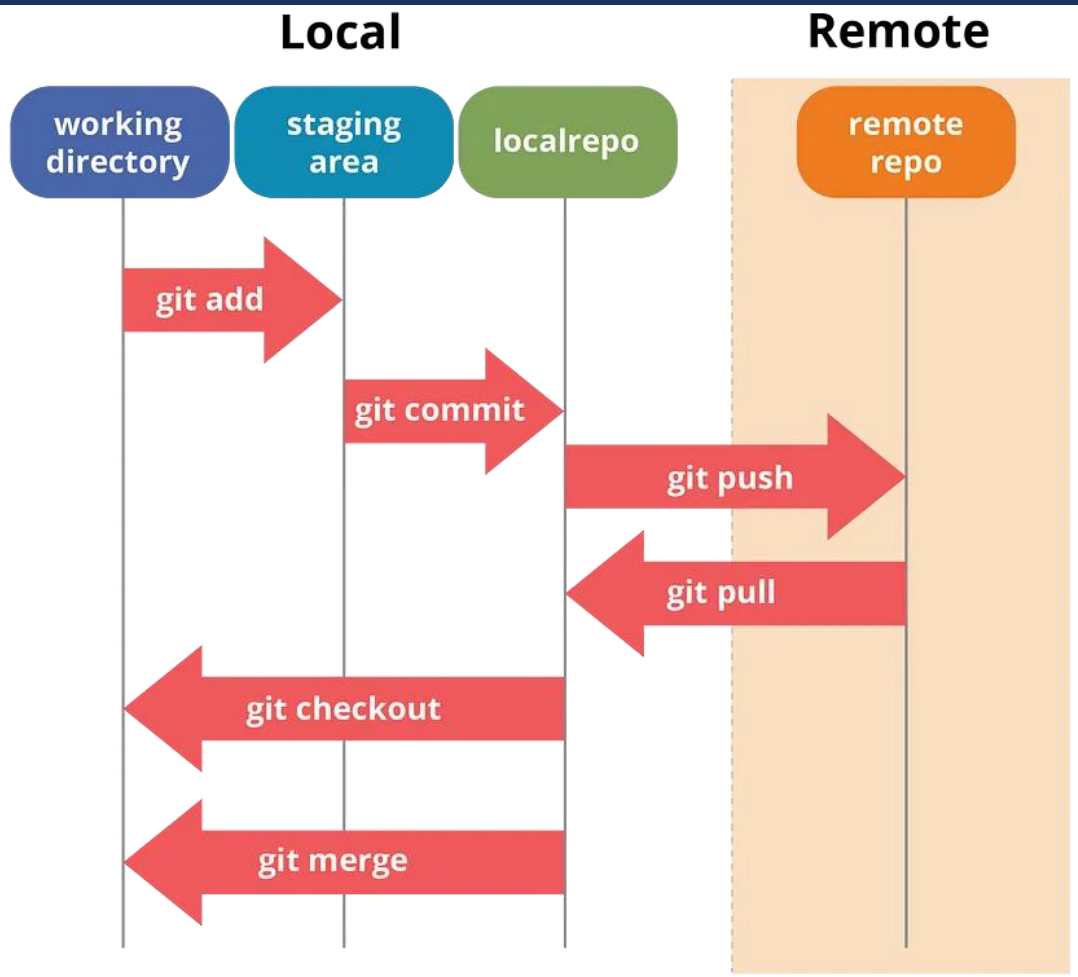
Git Concepts

- Git works on **Commits**.
- A Commit is very simply **a snapshot of all changes in the local repository** (more correctly, the working directory).
- Each Commit has a unique Commit ID.
- A Commit ID is a unique SHA-1 hash that is created whenever a new commit is recorded.
- The most recent commit is called the **Head** (more correctly, it refers to the currently checked-out branch's latest commit).
- For now, we will only deal with **main**/ master **Branch**.
- We will extensively deal with Commit IDs for many operations on git (more on this later).

Source: https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383

Git Concepts

1. **Add** a new file to the local Repo.
2. Do the required changes to the file.
3. **Commit** the file to the local repo.
4. **Pull** the changes from the remote repo.
5. **Push** the local changes to the remote repo.



Source: https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383

Scenario Revisited

- Refer back to Amith's version controlling method in the beginning. It could,
  - **Revert** to a past version.
  - **Compare & Merge** two different versions of files.
  - **Pull** changes from a remote location to anywhere.
  - **Push** changes done from anywhere to a secure remote location.
  - Use the **Log file** to keep track of all the events.

- All these actions can be easily accomplished using Git.
  - Revert – git revert (this reverts a commit that was made to the repository).
  - Compare & Merge – Automatically handled by git. No user intervention required most of the time (unless a conflict occurs).
  - Push and Pull – Update the **Upstream** and get updates from upstream.
  - Git has many ways of automatically keeping track of all the events (e.g.: git status, log, blame etc.)

Next week

- Branching
- Best Practices
- Pull Requests
- Git Workflows

References

1. Pro Git - Scott Chacon and Ben Straub [Free eBook]
2. Dangit, Git!?! [Git quick reference]
3. Reading 5: Version Control [MIT Lecture]
4. Distributed Version Control [Wikipedia]
5. History of Version Control Systems

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

VERSION CONTROLLING & WORKFLOWS WITH GIT - II

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

Learning outcomes

- After completing this lecture, you will be able to,
  - Apply the basic concepts of Git for version controlling
  - Apply an efficient branching strategy for a team
  - Describe what a git workflow is and why a workflow is important.
  - Apply a simple git workflow to your own projects.

Contents

- Recap from the previous week
- Git branching
  - The basics
  - How do we create a branch on git?
  - Workflows (Branching strategies)
  - An Example (GitFlow)
  - A Simpler Git Workflow
    - The GitHub Flow
    - Pull Requests (PRs)
  - Best practices for branching
- Git general best practices
- Additional topics
- Summary

From the previous week

- Version Controlling: Scenario Discussion
- A bit of history on Version Controlling Systems (VCS)
- Git Basic Concepts
  - Repositories – Local and Remote
  - Each user has their own local repository
    - This local repository can be obtained by,
      - Copying from another user (P2P)
      - **Cloning** it from a remote repository (on GitHub, BitBucket etc.)
      - Initializing their own repository.
  - Commits
  - Basic git operations (git add, commit, push, pull)

Git Branching: The basics

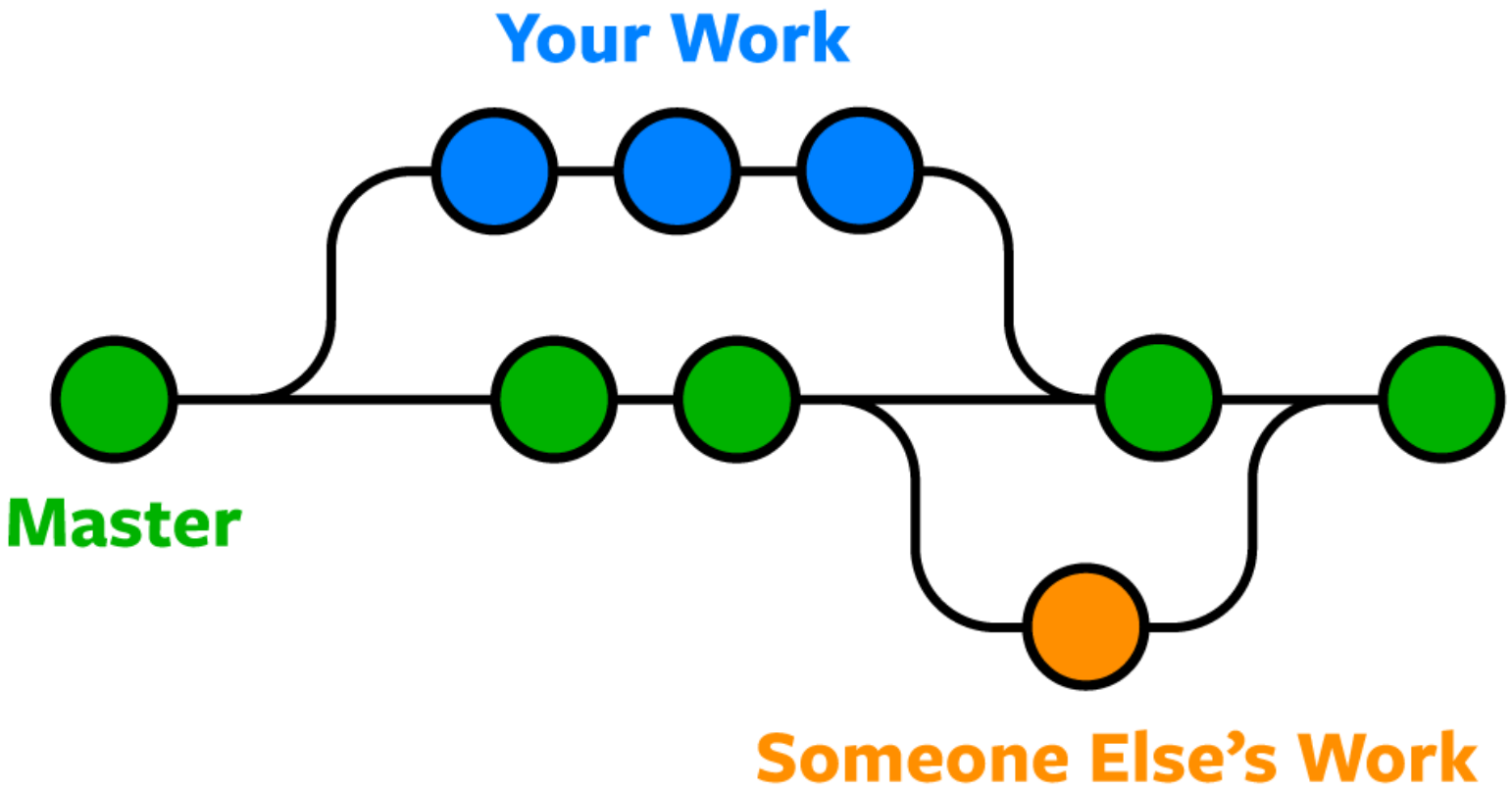
- Think of a tree. What is a branch in a tree?
  - An offshoot of the trunk of a tree.
- What is a branch in Git?
  - The default branch in Git is the ***master*** branch (It is like the trunk of a tree).
  - Very simply put, a branch is a detour you make from the main path of development.

Source: https://www.rainforest-alliance.org/wp-content/uploads/2021/06/kapok-tree-profile-1.jpg.optimal.jpg

Git Branching: The basics

- Why is branching needed?
  - Branching lets developers work independently inside a single code repository.
    - E.g., In product X, Amith works on feature A, while Binesh works on feature B.
    - If both have their code in just their local repositories, then no problem.
    - Now, it is a good practice to push your changes to the remote repository regularly.
      - What sort of problems can be expected when they do that?
        - **Merge Conflicts!**
  - Any solution to the above problem?
    - Yes, let each of them work on their own branch.

Git Branching: The basics



Example on git branching. Source: https://www.nobledesktop.com/learn/git/git-branches
SenthINC

Git Branching: The basics

- Why is branching needed? (continued)
  - Allows a developer to switch among different tasks easily.
    - Suppose while Amith is working on feature A, he gets a request to attend to a major issue in the product X.
      - If Amith is working on his local master branch, he will now have to,
        - Undo all the changes he did.
        - Fix the issue.
        - Restart the work on feature A.
      - If Amith had been working on a separate branch even if he was on his local repository, he could have,
        - Stopped the work on the feature and switch back to master.
        - Created new branch for the **"hotfix"** and work on it.
        - Pushed the hotfix branch to the remote repo (and merge to remote master)
        - Switched back to his feature branch and continue where he left off.

Git Branching: The basics

- Referring to the previous scenario,
  - Using git branches during development time is clearly more efficient rather than not using any.
  - Using branches is a must for collaborations with multiple developers.
  - Whatever you do on a branch is isolated from the rest of the code – great when you want to experiment!
  - It is not a bad idea to use branching for individual projects too (see the second bullet point on why branching is needed).
- Branching on Git is a very lightweight operation. Therefore, use it whenever possible.

How do we create a branch on Git?

- Very simple.
  - *git branch <branch_name>* - just creates the branch based on your current branch.
  - *git checkout <branch_name>* - checks out the branch.

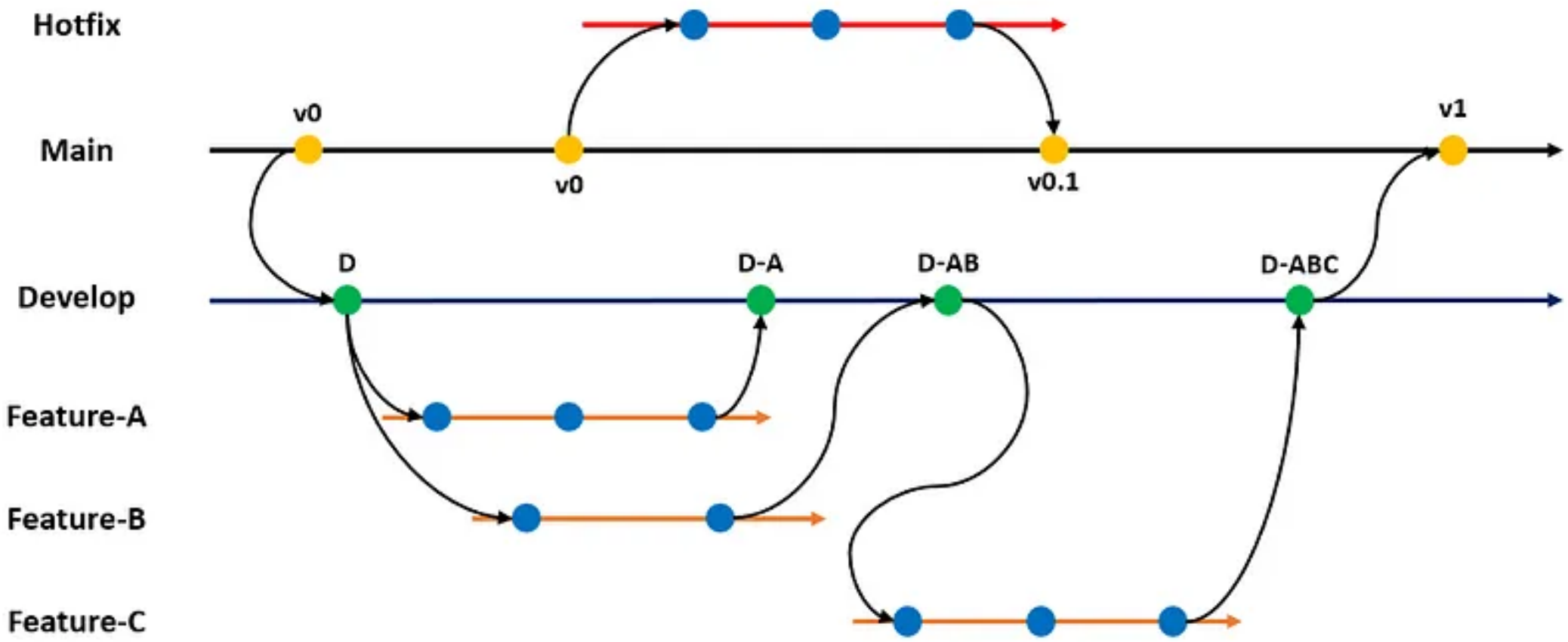
OR

  - *git checkout –b <branch_name>* - creates the branch, and switches to the new branch (checks it out) from your current branch immediately.

Workflows (Branching Strategies)

- Git is very flexible when it comes to how-to branch. There is no one standard method, but some well accepted how-to branch 'recipes' are there.
- They are called workflows or branching strategies.
  - A Git workflow is a recipe or recommendation for how to use Git to accomplish work in a consistent and productive manner.
- Some popular workflows are,
  - GitFlow
  - GitHub Flow
  - GitLab Flow
  - Trunk Based Development (goes with CI/ CD)

Git Branching: An Example (GitFlow)



Source: https://betterprogramming.pub/git-branching-strategy-for-better-team-collaboration-aacb5f235d05

A Simpler Git Workflow

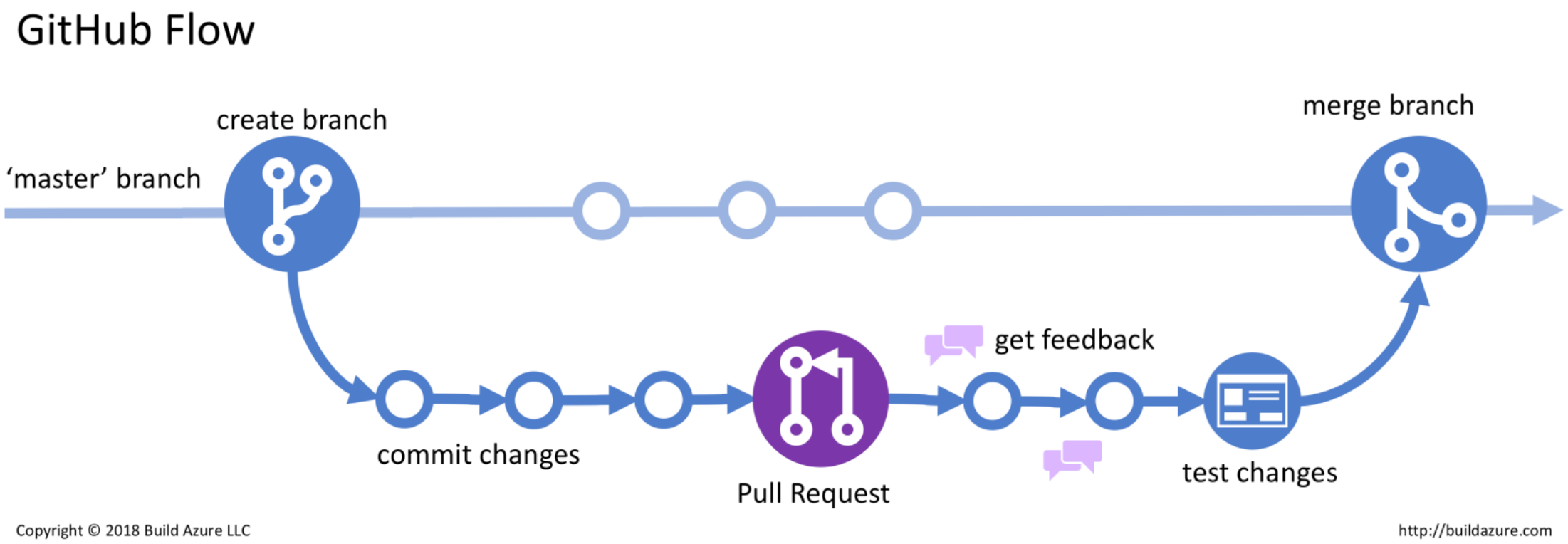
- The GitFlow is very comprehensive.
  - Can become unnecessarily complex for small projects.
- It is more suited to Enterprise grade software development efforts.
  - For most use cases, it is overkill.
- Therefore, a simpler alternative is preferable for most use cases.
  - One such workflow is the GitHub Flow which can be used by anybody.

The Github Flow

- This is a GitHub based simple workflow.
  1. Create a new feature branch from the main (master) branch.
  2. Make the necessary changes to the feature branch – continue until you think your feature is ready.
  3. Once you think the feature is done, ask for feedback from the other developers – create a Pull Request (PR) for this.
  4. Address their review comments received on the Pull Request (PR).
  5. Merge your Pull Request (PR) to the main branch.
  6. Delete your feature branch.

Refer the official documentation here.

The Github Flow



Source: https://build5nines.com/introduction-to-git-version-control-workflow/

Pull Requests (PR)

- A Pull Request (PR) is used to propose a new change to a repository (usually to the main branch).
- Usually, the proposed changes are in a feature branch, which ensures that the main branch will only contain code which is tested and working.
- The other collaborators will have to verify that the proposed change (PR) is OK first.
- If the PR is OK, they will accept it. Or else, they will suggest changes until it is acceptable.
- Once the PR is accepted, the changes in the feature branch are merged automatically to the main branch, bringing the proposed change to the main branch.
- Find the official documentation on Creating PRs here.

Git Branching: Best practices

1. Create a branch for each feature. Do not work on multiple features in a single branch.
2. Ideally, only one person should work on one feature branch.
3. Feature branches should be short-lived. They **should not exist** once the feature is complete.
4. Delete the local and remote feature branch after merging to the master branch (see point 3).

Git Branching: Best practices

5. Merge early and often to avoid merge conflicts.
6. Keep the master branch clean. Only *production ready* code should be in the master branch. It is better to **avoid** working directly on the master branch.
7. Use a good naming convention to name your feature branches. Everyone should stick to the same naming convention.
   - E.g.:
     - feature/<username>/task-name-in-lowercase-spaces-replaced-with-dashes
       - feature/vishan.j/awesome-feature-x-to-our-app

Git general best practices

1. Make small, incremental changes.
   - Don't make huge changes in one go. If you have a big feature to add, break it down to smaller features and add one at a time.

- Keep commits atomic.
  - It is better if each commit is focused on one part of the feature. This will minimize the damage if you must revert a commit.
- Commit often.
  - Commit your changes often. Even if the work is not done.
- Pull the changes in origin/main branch before you create a new branch from your local/main.
  - This ensures that the new feature branch is created with all the latest changes in master. This reduces the possibility of merge conflicts later.

Git general best practices

5. Push local feature branch commits to Remote often.
   - Even if it is an unfinished feature, push it. That's good for safekeeping.

- ☐ Use branches.
  - See the section on branching for a comprehensive set of reasons.
- ☐ Write good commit messages.
  - Make the lives of other Software Engineers, DevOps engineers, QA engineers easier.
- ☐ Select one branching strategy (Git Workflow) and stick with it.
  - Select one that suits you and use it well.

Summary

- Git branching
  - The basics
  - How do we create a branch on git?
  - Workflows (Branching strategies)
  - An Example (GitFlow)
  - A Simpler Git Workflow
    - The GitHub Flow
    - Pull Requests (PRs)
  - Best practices for branching
- Git general best practices

Additional Topics

Read on these topics to be up to date on some current trends.

- Continuous Integration and Continuous Deployment (CI/ CD).
- Compare Trunk based development workflow with GitFlow workflow.
  - Understand when to use which.
- Read on Pull Requests.
- Read on Git Rebase and see why it's a better alternative to Git Merge.

References

1. Pro Git - Scott Chacon and Ben Straub [Free eBook]
2. Dangit, Git!?! [Git quick reference]
3. Branch in Git
4. What Are the Best Git Branching Strategies
5. 10 Git Branching Strategy Best Practices
6. Trunk-based Development vs. Git Flow
7. How to Use Git and Git Workflows – a Practical Guide

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

WEB APPLICATION ARCHITECTURE – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

Learning outcomes

- After completing this lecture, you will be able to,
  - Describe the main components of three tier architecture for web development.
  - Identify some suitable technologies to develop the presentation layer and the application layer of a web application.
  - Apply suitable architectural components to design the presentation layer and the application layer of a Three tier web application.

Contents

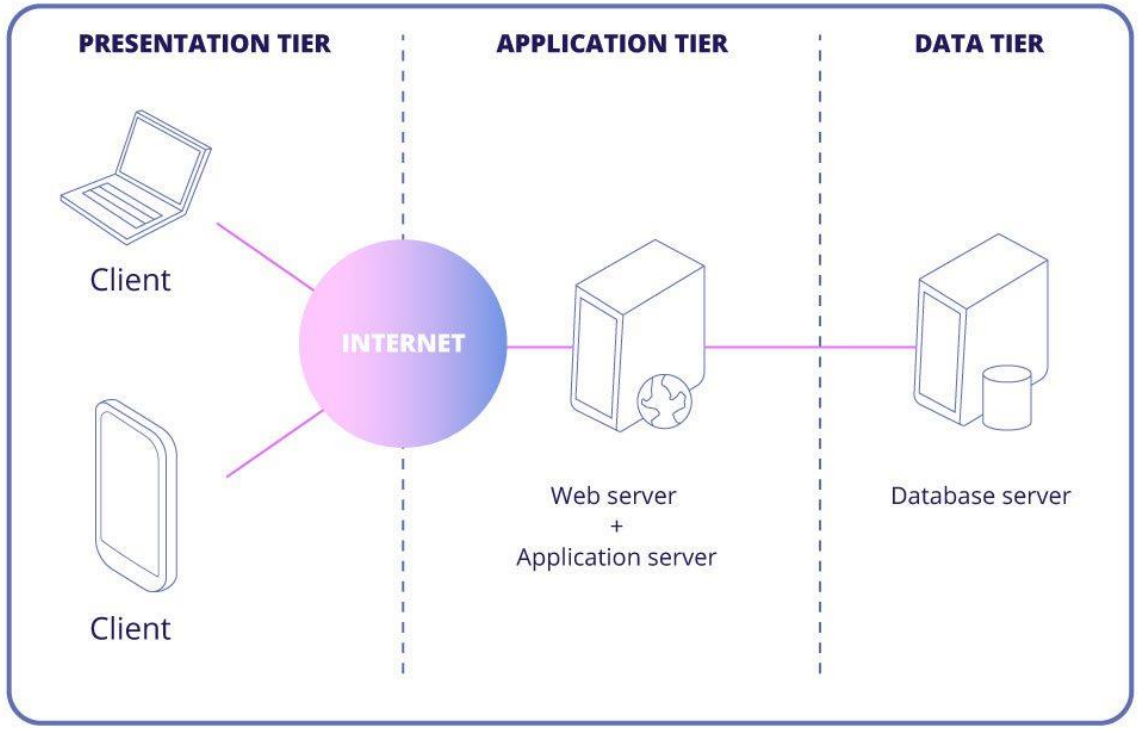
- Website vs Web Application
- Three Tier/ Three Layer Architecture
- Presentation Layer
  - Some technologies
  - Architecting the Presentation Layer
- Application Layer
  - Some technologies
  - Application Programming Interface (API)
  - Architecting the Application Layer
  - Big Ball of Mud: an architectural anti-pattern
- Other Helpful Technologies
- Summary

Website vs. Web Application

- Q: What is the difference between a Website and a Web Application?
  - A Website provides static content which can be consumed by the end users.
  - A **Web Application** is designed for **interaction** with the end users.

- Our focus is on **Web Applications**.

Three tier architecture



Source: https://mobidev.biz/blog/web-application-architecture-types

mobidev

Three tier architecture

- Modern web applications are engineered based on this architecture.

- Has three tiers.
  - Presentation tier/ layer
  - Application tier/ layer
  - Database tier/ layer

- The most popular form of n-tier (multitier) architecture.

Three tier architecture

- Three tier architecture is a good choice for the development of web applications.

- Reasons?
  - Each tier/ layer runs on its own infrastructure.
  - Each tier/ layer can be developed independently/ in parallel by different teams.
  - Allows updating and scaling up/ down each tier independently without impacting the other tiers.
  - Better security when compared with two tier architecture (due to the isolation of data and logic from the presentation tier).

Presentation layer

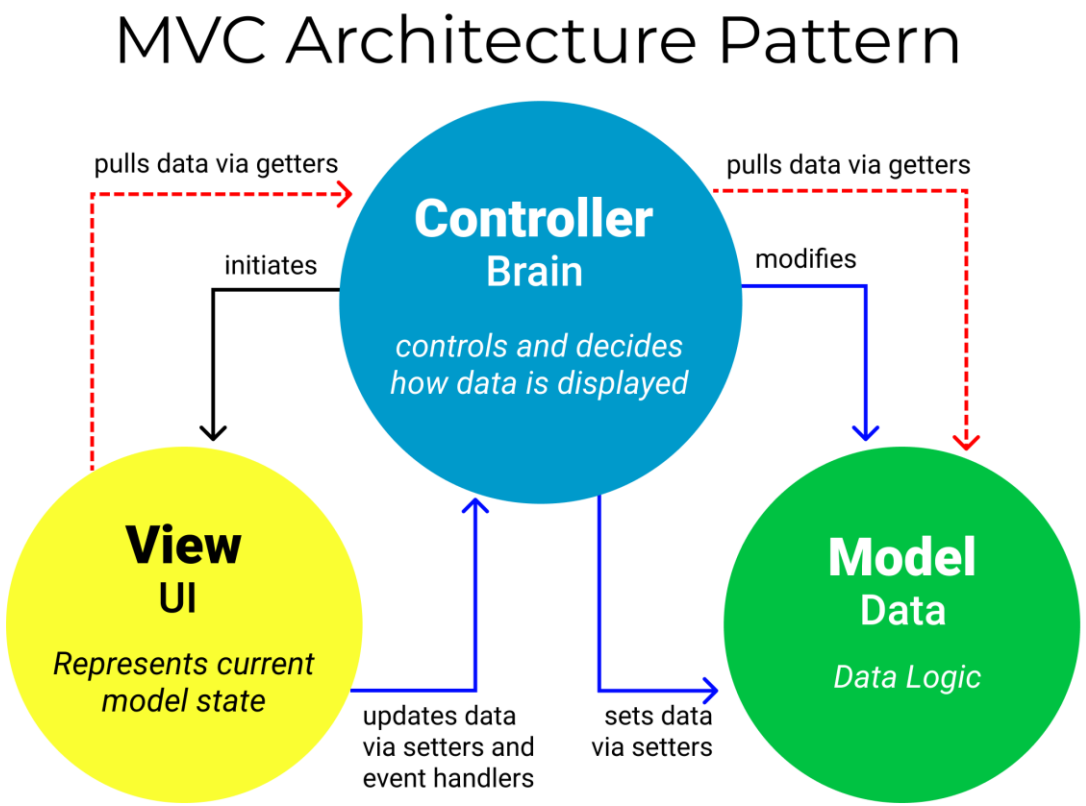
- Also called the client side or the frontend.
- Technologies used to implement include,
  - HTML/ CSS/ JavaScript
  - React.js
  - Angular
  - Vue.js
  - Some legacy technologies
    - jQuery and AJAX

Architecting the Presentation Layer

- Some well known examples
  - Model-view-controller (MVC)
  - Model-view-viewmodel (MVVM)

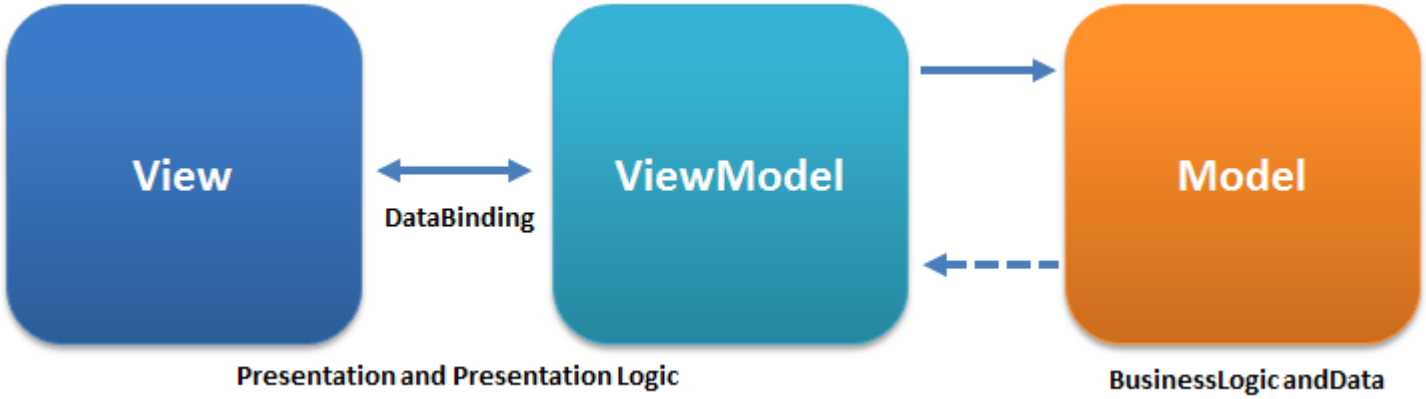
- Some more examples
  - Model-view-presenter (MVP)
  - Component Architecture
  - Micro frontends

Architecting the Presentation Layer: MVC



Source: https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/

Architecting the Presentation Layer: MVVM



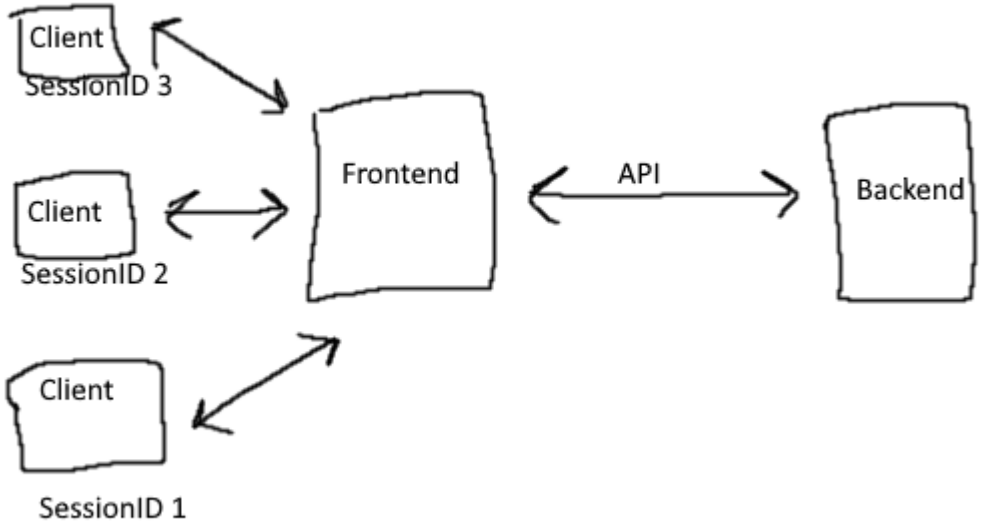
Source: https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93viewmodel

Application Layer

- Also called the server side or the backend.
- Technologies used to implement include but not limited to,
  - Programming languages and frameworks
    - Java (frameworks: Spring/ Spring Boot)
    - Node.js (frameworks: express.js/ Koa)
    - Python (frameworks: Django/ Flask)
    - PHP (frameworks: Laravel, Symphony)
    - .Net Framework
  - Virtual Machines (VMs)
  - Serverless Computing
  - Containers

Application Layer

- How does the Backend and the frontend communicate?



Source: https://stackoverflow.com/q/71033687

Application Layer: APIs

- Application Programming Interface (API)!

- What is it?
  - A software interface which facilitates communication between two or more applications.
  - Abstracts away the complexities of application integrations.
  - A contract of services offered.

Application Layer: APIs

- Benefits of APIs?
  - developers don't have to create everything from scratch - can use existing functions exposed as an API resulting in more productivity.
  - APIs significantly reduce development costs by reducing development efforts.
  - Improves collaboration and connectivity across the ecosystem.

Application Layer: APIs

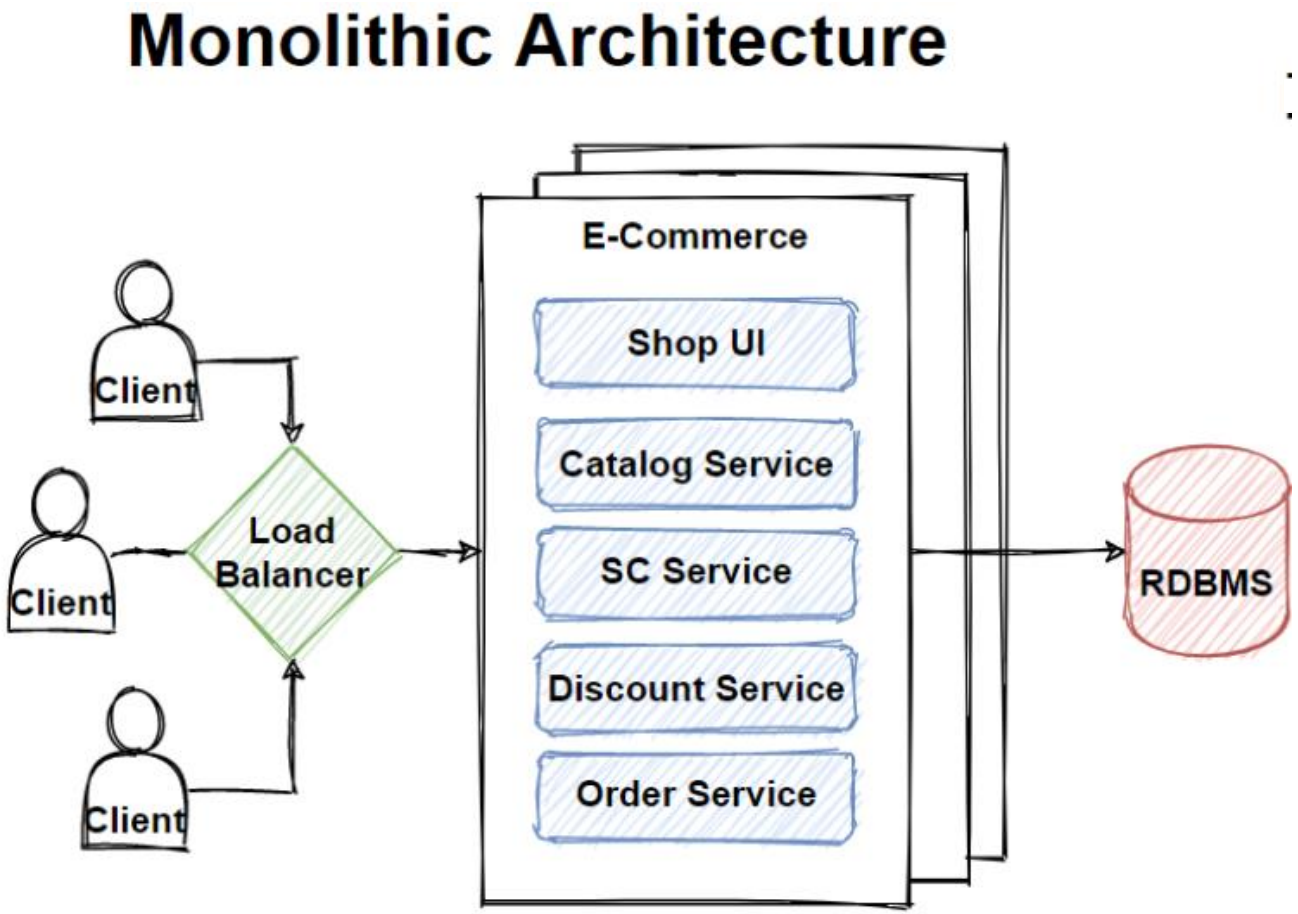
- Different types of APIs?
  - Representational State Transfer API (REST API)
  - Simple Object Access Protocol API (SOAP API)

More on these in a separate lecture.

Architecting the Application Layer

- Some well known examples
  - Monolithic Architecture
  - Microservices Architecture

Architecting the Application Layer: Monolithic



Source: https://medium.com/design-microservices-architecture-with-patterns/monolithic-architecture-is-still-worth-at-2021-98bfc112dc24

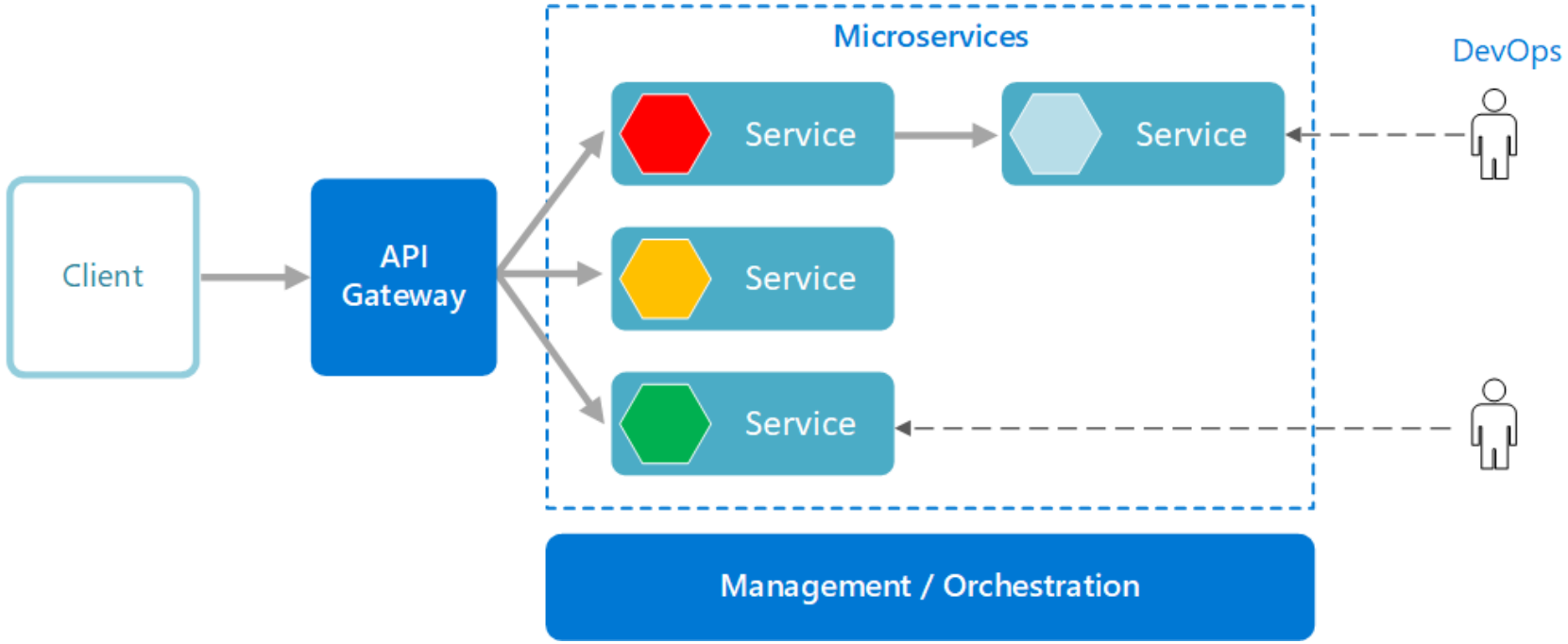
Architecting the Application Layer: Monolithic

- Drawbacks in Monolithic Architecture?
  - Slower development speed due to **local complexity.**
  - Can't scale individual components.
  - Reliability – if there's an error in any module, it could affect the entire application's availability.
  - Barrier to technology adoption – any changes in the framework or language affects the entire application, making changes often expensive and time-consuming.
  - Lack of flexibility – a monolith is constrained by the technologies already used in the monolith.
  - A small change to a monolithic application requires the redeployment of the entire monolith.

Architecting the Application Layer: Microservices

- "Microservices architecture is an approach in which a single application is composed of many loosely coupled and independently deployable smaller services." (Source – IBM)

Architecting the Application Layer: Microservices



Source: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices

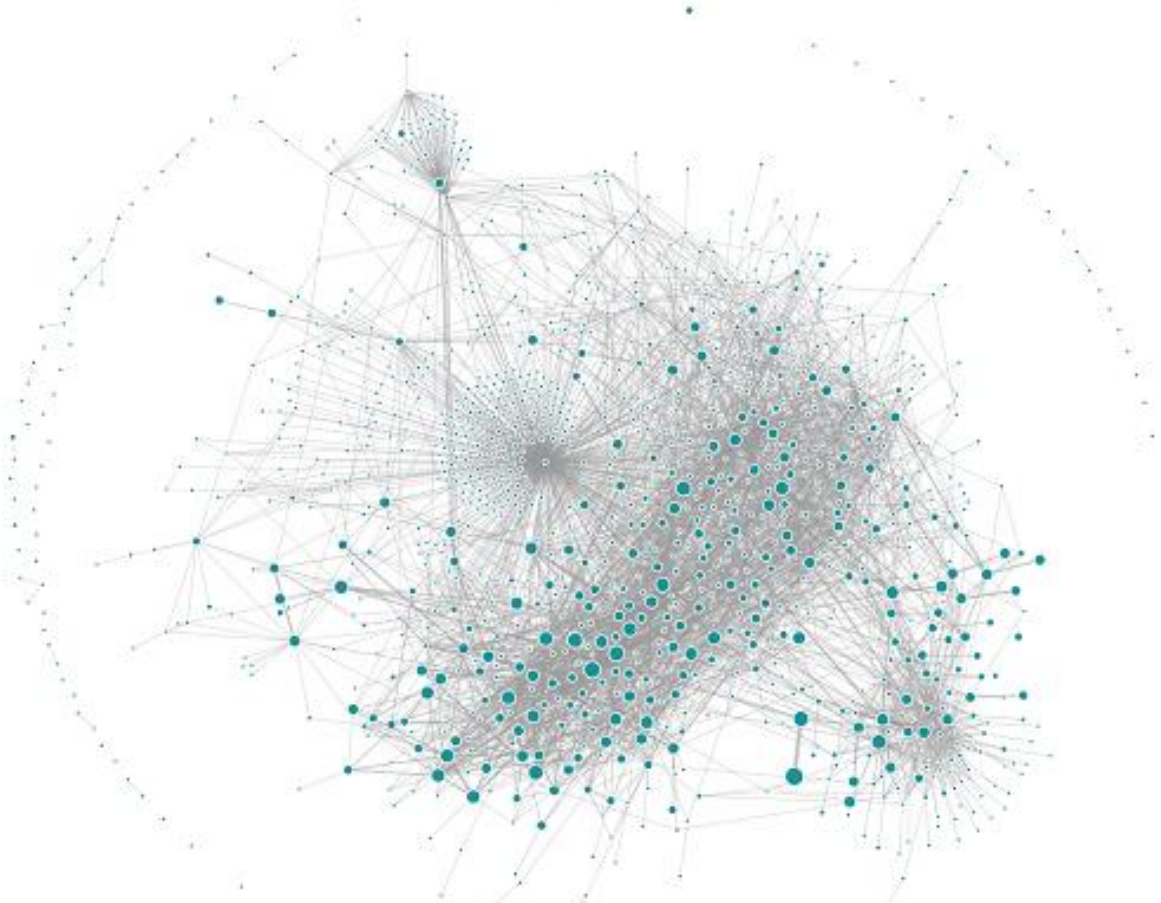
Architecting the Application Layer: Microservices

- Some benefits of Microservices
  - Microservices are **small, independent, and loosely coupled** (therefore, a single team can write/ maintain a single service).
  - Services can be **deployed independently** (can update an existing service without rebuilding/ redeploying the entire application).
  - Can **scale** each service independently
  - High **reliability**
  - Supports **polyglot programming**

Architecting the Application Layer: Microservices

- Is everything great with Microservices?

Architecting the Application Layer: Microservices



Uber's microservice architecture circa mid-2018

Source: https://www.uber.com/en-LK/blog/microservice-architecture/

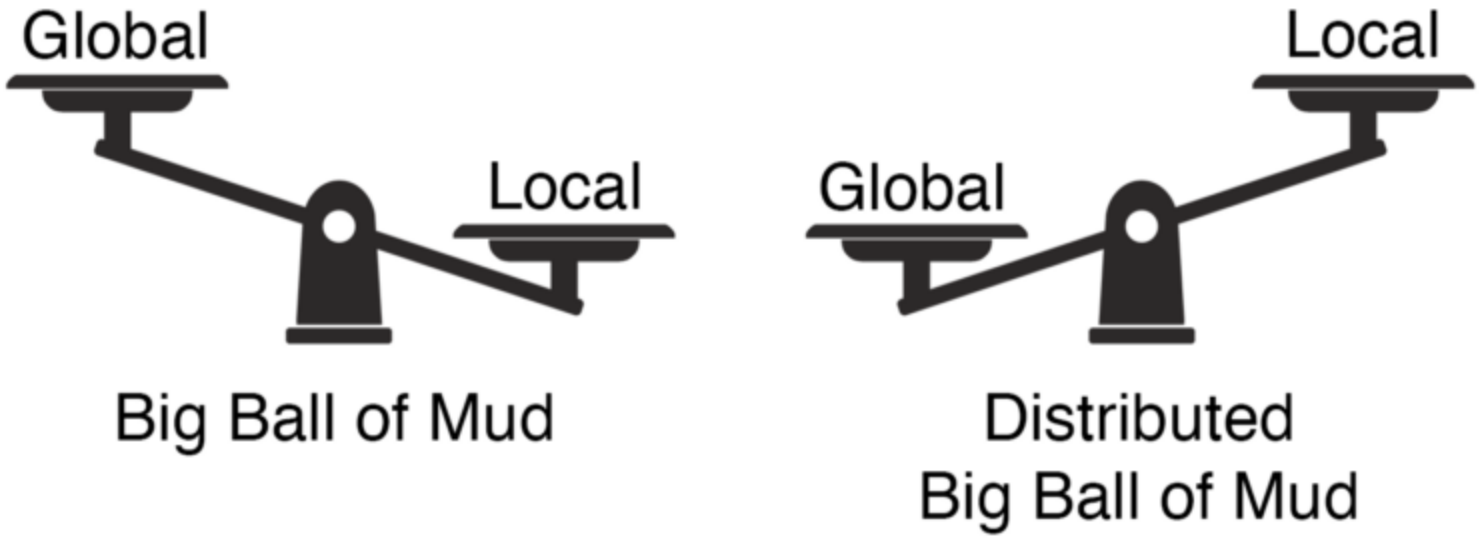
Architecting the Application Layer: Microservices

- **High global complexity** is the biggest problem when in comes to Microservices.
  - *Local complexity* depends on the **implementation of a service**; *global complexity* is defined by the **interactions and dependencies** between the services.
- Managing complexity is a constant challenge in Microservices.

Architecting the Application Layer: Microservices

- What are some other downsides of Microservices?
  - High infrastructure costs
  - Debugging challenges
  - A lot more…

Big Ball of Mud: an architectural anti-pattern



Source: https://www.doit.com/untangling-microservices-or-balancing-complexity-in-distributed-systems/

Other helpful technologies

- API Gateways (authentication, throttling, orchestration, caching, statistics etc.)
- Content Delivery Networks (CDN)
- Caching Tools (e.g., Redis)
- Load Balancers
- Message Queues
- Cloud Computing
  - Cloud Storage
  - Virtual Machines (VMs)
  - Many more resources…

Some interesting topics

- Component Based Architecture
- Micro Frontends – Microservices for the frontend
- BFF: Backend For Frontend also known as BIF: Backend In Frontend

References

1. Web application vs. website: finally answered
2. What is three-tier architecture?
3. SOAP APIs vs REST APIs
4. Micro Frontends: What are They and When to Use Them?
5. Model–view–presenter
6. An In-Depth Guide to Component-Based Architecture: Features, Benefits, and more
7. The Complete Guide To BFF (Backend For Frontend)
8. Microservices vs. monolithic architecture
9. Untangling Microservices or Balancing Complexity in Distributed Systems
10. Big Ball of Mud - The Daily Software Anti-Pattern

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

REST APIS

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

Learning outcomes

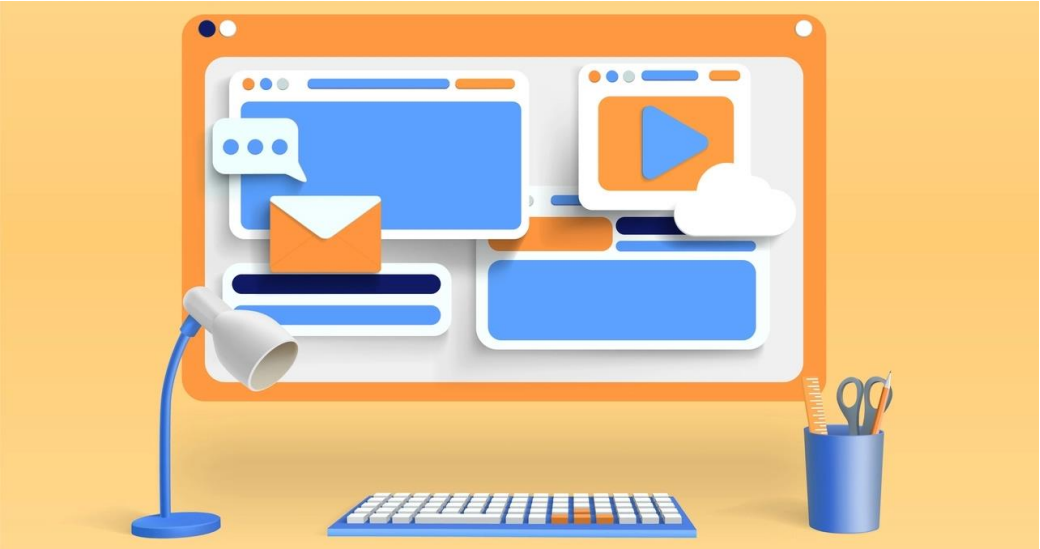
- After completing this lecture, you will be able to,
  - Describe the six REST architectural constraints.
  - Apply the knowledge gained in this lecture in designing proper REST APIs.
  - Evaluate if an API is a REST API or not.
  - Evaluate a given API to identify how much it conforms to the REST constraints using the Richardson Maturity Model.

Contents

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
  - Examples
- REST
  - Six Architectural Constraints of REST
  - True REST vs. RPC (Remote Procedure Calls)
  - Richardson Maturity Model
- Some Facilitators of REST
    - HTTP, JSON, HAL
- Best Practices

Consuming web resources - Human vs. programs

- How do you consume web resources?
  - Web browsers
  - Web applications
  - Mobile applications etc.



Source: https://illuminatingfacts.com/a-timeline-of-the-rise-and-fall-of-popular-web-browsers/

Consuming web resources - Human vs. programs

- How do programs consume web resources?
  - Web APIs!
  - API stands for **Application Programming Interface**

Source: https://stileex.xyz/en/difference-computer-program-software/

Application Programming Interface (API)

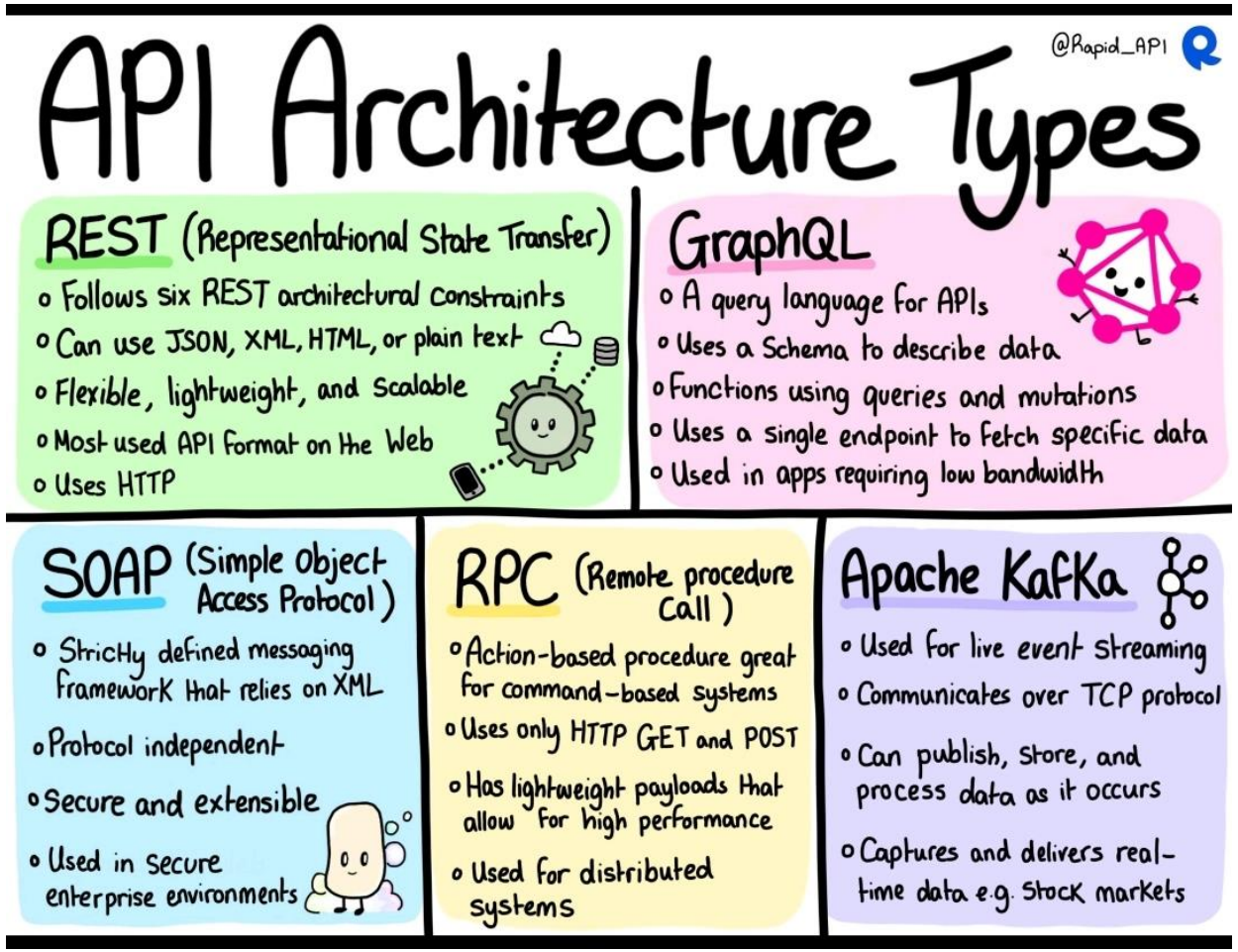
- What is an API?
  - A software interface which facilitates communication between two or more applications.
  - Abstracts away the complexities of application integrations.
  - A contract of services offered.

Application Programming Interface (API)

- Examples:
  - RPC (Remote Procedure Calls)
  - CORBA Implementations (Common Object Request Broker Architecture)
  - SOAP (Simple Object Access Protocol)
  - **REST** (Representational State Transfer)

Mostly legacy technologies

Application Programming Interface (API)



Source: https://ee.linkedin.com/posts/theo-borek-higgins_devops-api-devopslearn-activity-7028421145682944000-0YgL

REST (Representational State Transfer)

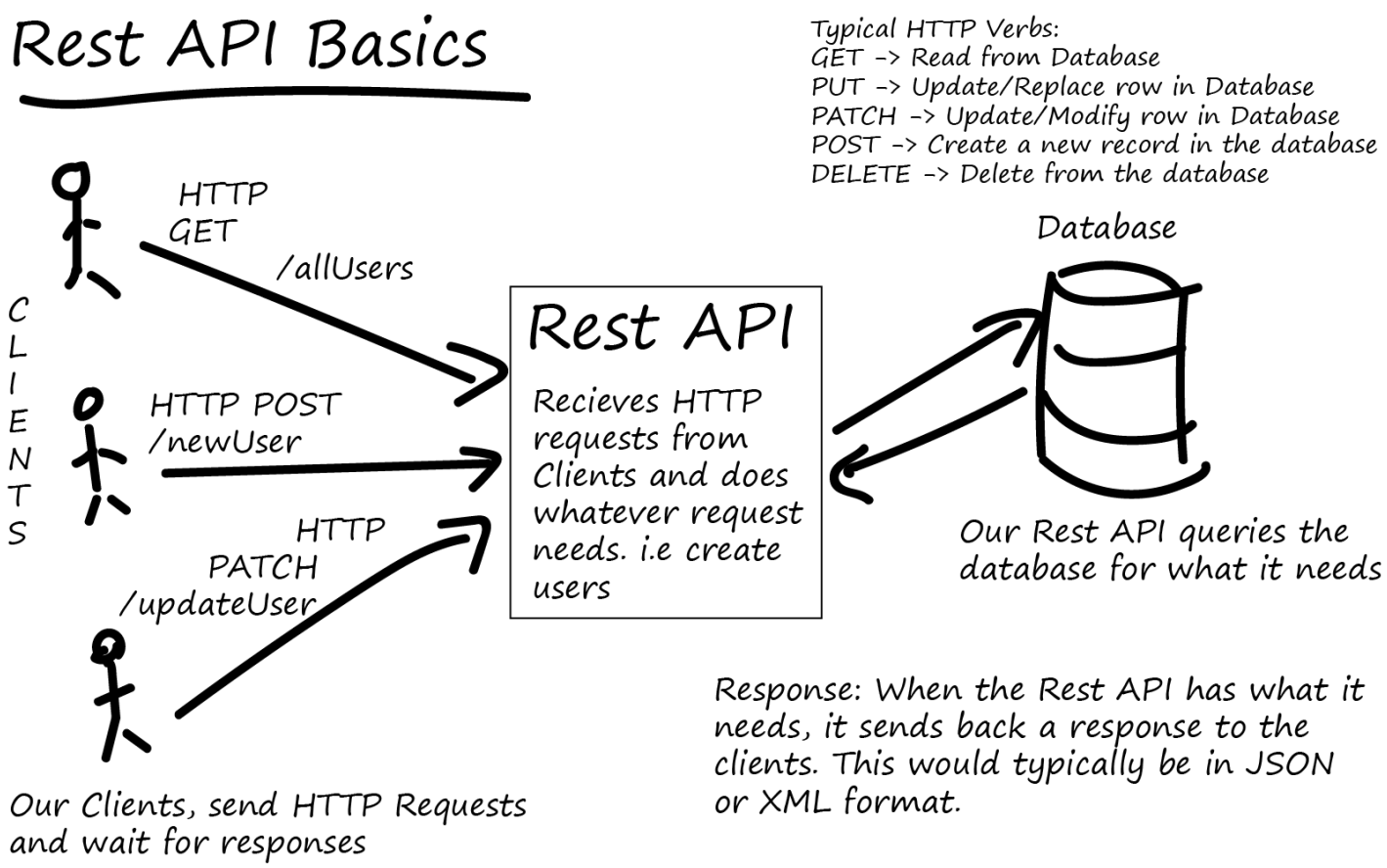
- What is REST?
  - an **architectural style** for creating web services.
  - Introduced by Roy Fielding in 2000 in his now famous Doctoral dissertation.
  - Arguably the most popular approach to creating Web APIs now.

Source: https://medium.com/@liams_o/15-fundamental-tips-on-rest-api-design-9a05bcd42920

REST (Representational State Transfer)

- Why is REST Popular?
  - Just an **architectural style** (not a technology or a full-blown protocol like SOAP)
  - If the API aligns with the six REST constraints, it is a REST API
  - No strict rules, very flexible
  - Simple to use and learn compared with previously used approaches (e.g., SOAP/ CORBA etc.)
  - Runs on HTTP Protocol

REST (Representational State Transfer)



Source: https://tutorialedge.net/software-eng/what-is-a-rest-api/

REST (Representational State Transfer)

- An example
  - A proper REST response to a GET call to http://localhost:8080/employees/1 REST endpoint.


```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```


Source: https://spring.io/guides/tutorials/rest/

REST (Representational State Transfer)

- Design Principles
  - REST is designed to facilitate **strong decoupling** between the client and the server to facilitate internet-scale usage.
  - Follows a Request/ Response style of communication.
  - Some Terms:
    - Client – the consumer of services via the API
    - Server – the provider of services and the API that exposes these services
    - Resource – any content that the server responds with to a client request

REST (Representational State Transfer)

- Design Principles (contd.)
  - When a **client** requests for a **resource**, the **server** will respond with
    - the **representation** of the resource in its current **state** (Usually JSON or XML)
    - and relevant **hypermedia links (URIs)** that can be used to change the **state** of the resource

REST: Six architectural constraints

- Client-server architecture
- Stateless
- Cacheable
- Uniform Interface
- Layered system
- Code on demand (Optional)

Let's examine each constraint.

REST: Six architectural constraints

- Client-server architecture
  - Maintain the clear separation of the client and the server,
    - To promote portability of the user interface
    - To promote low coupling between the client and the server
    - To improve scalability of the server components
    - To allow clients and servers grow independently

REST: Six architectural constraints

- Stateless
  - Communication between clients and server must be stateless in nature
  - Each request from client to server must contain all the information necessary to understand the request and cannot take advantage of any stored context on the server.
  - Session state is kept entirely on the client.
  - Improves visibility, reliability, and scalability.

REST: Six architectural constraints

- Cacheable
  - Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable.
  - If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
  - Caching can eliminate requirement for some interactions thereby improving efficiency, scalability, and user-perceived performance.

REST: Six architectural constraints

- Uniform Interface
  - Implementations are decoupled from the services they provide, which encourages independent evolvability.
  - Continued in the next slide…

REST: Six architectural constraints

- Uniform Interface
  - Has four sub-constraints.
    - **Self-descriptive messages** - Each message going back and forth between the client and the server should contain enough information to process the message.
    - **Identification of resources** - Individual resources are identified in requests/ responses. This is done through URIs (Uniform Resource Identifiers). The resources themselves on the server are conceptually separate from the representations that are returned to the client.
    - **Resource manipulation through representations** - When a client holds a representation of a resource, including any metadata attached, it has enough information to modify or delete the resource's state
    - **Hypermedia as the engine of application state** (**HATEOAS**) – continued in the next slide…

REST: Six architectural constraints

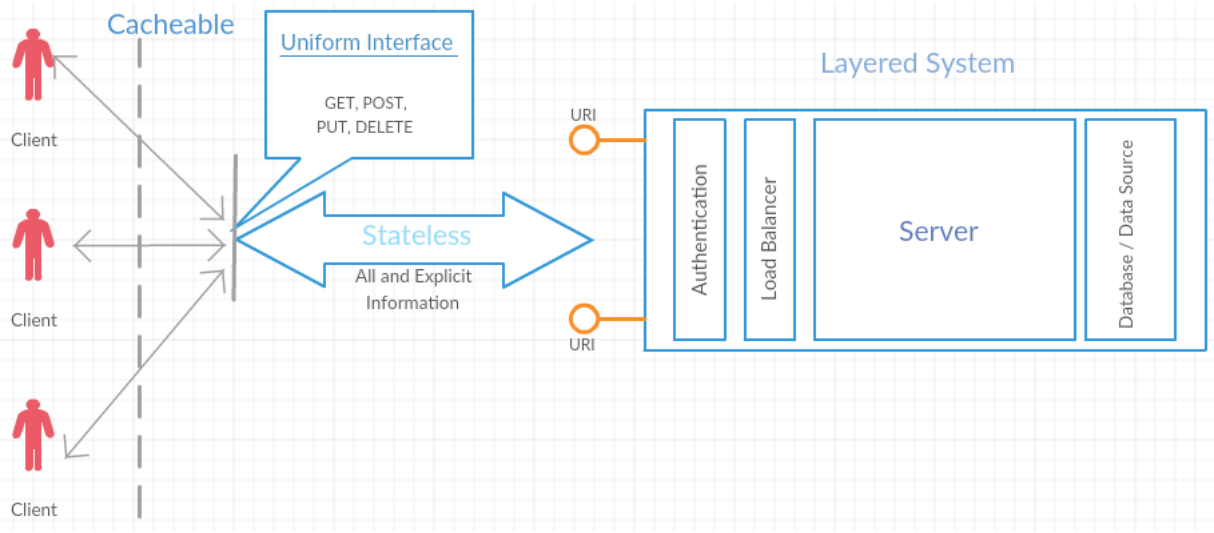
- Uniform Interface (continued…)
  - **Hypermedia as the engine of application state (HATEOAS)** - (Hey-tee-oh-s)
    - Having accessed an initial URI for the REST application a REST client should then be able to use server-provided links dynamically to discover all the available resources it needs.
    - As access proceeds, the server responds with text that includes hyperlinks to other resources that are currently available (Dynamic API).
    - There is **no need** for the client to be hard-coded with information regarding the structure of the server.
    - This is akin to a human user having to know only the main URL of a website. The rest of the URLs are discovered when consuming the site.

REST: Six architectural constraints

- Layered system
  - Each layer only interacts with the layers above and below them.
  - The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot "see" beyond the immediate layer with which they are interacting.
  - The layered system constraints enable network-based intermediaries such as proxies and gateways to be transparently deployed between a client and server using the Web's uniform interface.
    - A network-based intermediary will intercept client-server communication for a specific purpose. Network-based intermediaries are commonly used for enforcement of security, response caching, and load balancing.

REST: Six architectural constraints

- Layered system (continued…)



Source: REST API Design Rulebook (Mark Masse, Oreilly, 2011)

REST: Six architectural constraints

- Code on demand (Optional)
  - REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts.
  - This simplifies clients by reducing the number of features required to be pre-implemented.
  - This is the only optional constraint in REST.

True REST vs. RPC (Remote Procedure Calls)

- Pretty URLs like /employees/10 aren't REST.
- Merely using HTTP methods such as GET, POST etc. is not REST.
- Having all the CRUD operations laid out isn't REST.

- In such an API there is no way to know **how to interact with the service** without explicit help from the developers.
  - HATEOAS constraint is missing!

True REST vs. RPC (Remote Procedure Calls)

The response to a GET call to http://localhost:8080/employees/1 endpoint.

```
[
  {
    "id": 1,
    "name": "Bilbo Baggins",
    "role": "burglar"
  }
]
```

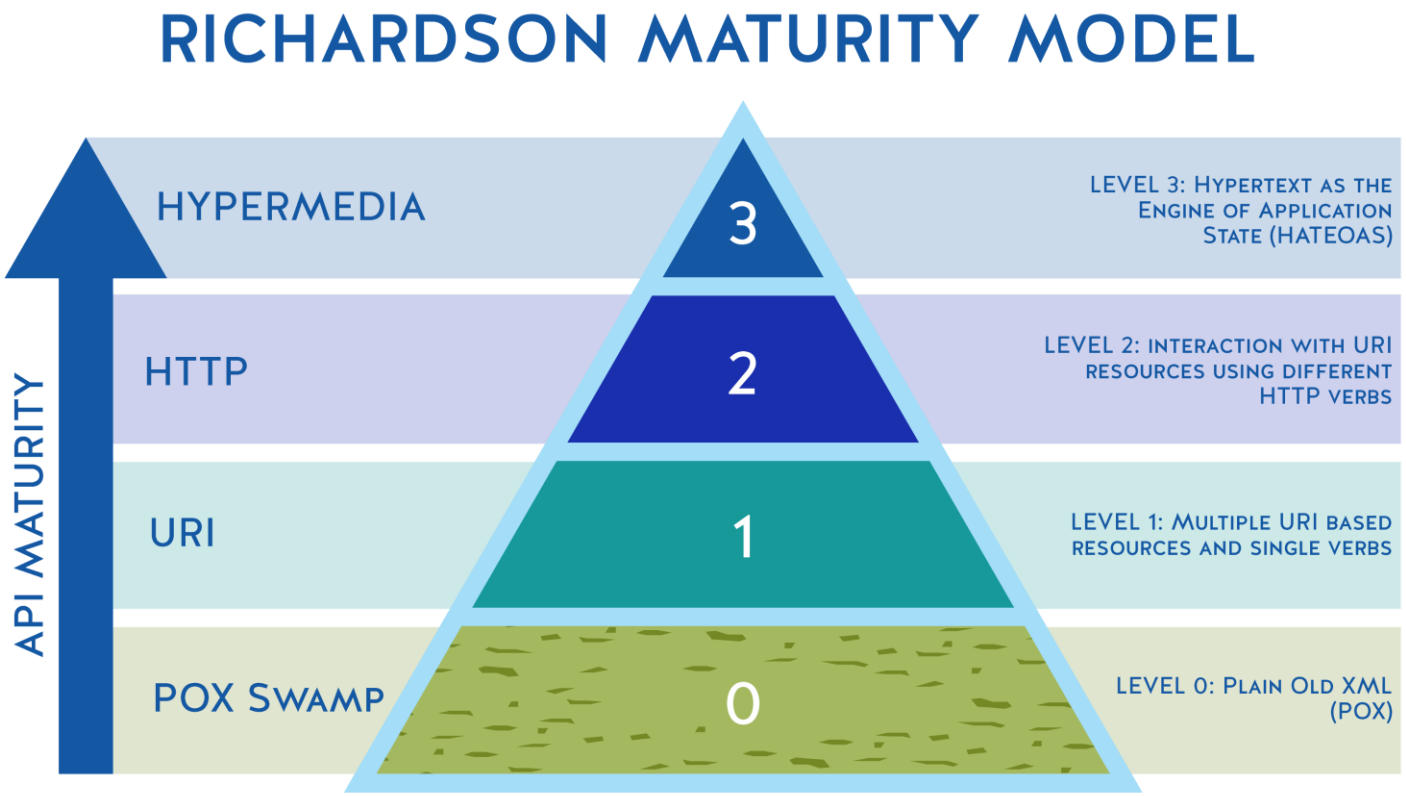
Just a JSON entity. No hyperlinks – Results in RPC

```
{
  "id": 1,
  "name": "Bilbo Baggins",
  "role": "burglar",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

RESTful representation of the same entity

Richardson Maturity Model

- It is a four-level scale that indicates extent of API conformity to the REST constraints.
- Proposed by Leonard Richardson.



Source: https://devopedia.org/richardson-maturity-model

Richardson Maturity Model

- Level-0: Swamp of POX
  - Usually exposes just one URI for the entire application. Uses HTTP POST for all actions, even for data fetch. SOAP or XML-RPC-based applications come under this level. POX stands for Plain Old XML. Could be JSON too.
- Level-1: Resource-Based Address/URI
  - Introduces resources and allows to make requests to individual URIs (still all typically POST) for separate actions instead of exposing one universal endpoint (API).
- Level-2: HTTP Verbs (Methods)
  - Fully utilize all HTTP commands or verbs such as GET, POST, PUT, and DELETE.
- Level-3: HATEOAS
  - Full maturity. Utilizes URI, HTTP Methods and HATEOAS.

Some Facilitators of REST

- HTTP
  - HTTP Methods – REST APIs utilize these to indicate the type of operation being performed.
    - GET – Reading a resource
    - POST – Writing a resource
    - PUT – Updating a resource
    - DELETE – Deleting a resource
  - HTTP Status Codes – REST APIs utilize these to denote the status of a particular request.
    - 2XX – Successful responses range (e.g., 200 OK)
    - 4XX – Client errors range (e.g., 404 NOT FOUND)
    - 5XX – Server errors range (e.g., 500 INTERNAL SERVER ERROR)
    - List of HTTP status codes: HTTP response status codes

Some Facilitators of REST

- HTTP (continued…)
  - HTTP Headers – These are used to pass along additional information/ meta data between the client and the server.
    - Content-type
    - Cache-control
    - Authorization
    - List of HTTP Headers: HTTP headers
- It is important that proper HTTP methods, Headers and Status Codes are used in REST requests/ responses.

Some Facilitators of REST

- JSON (JavaScript Object Notation) – A lightweight human readable format for storing and transporting data as key-value pairs.
- HAL (Hypertext Application Language) – An open specification that provides a structure to represent RESTful resources

```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

Source: https://spring.io/guides/tutorials/rest/

Some Best practices

- Use correct HTTP methods, headers and status codes according to the operation being performed.
- Accept and respond with JSON.
- Always use proper authentication (e.g., OAuth, JWT etc.)
- Give meaningful names to the URIs.
  - E.g.,
    - https://abcd.com/users/
    - https://abcd.com/users/2
- Keep the API consistent.

More Best Practices

- Self study activity
  - Read and follow this list of best practices from Microsoft: RESTful web API design
  - Make sure you apply these in your assignment as well!
- More hands-on experience on REST APIs during the practical session.

Summary

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
  - Examples
- REST
  - Six Architectural Constraints of REST
  - True REST vs. RPC (Remote Procedure Calls)
  - Richardson Maturity Model
- Some Facilitators of REST
    - HTTP, JSON, HAL
- Best Practices

References

1. An Introduction to Representational State Transfer (REST)
2. Architectural Styles and the Design of Network-based Software Architectures
3. Representational state transfer
4. Richardson Maturity Model
5. RESTful web API design

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK